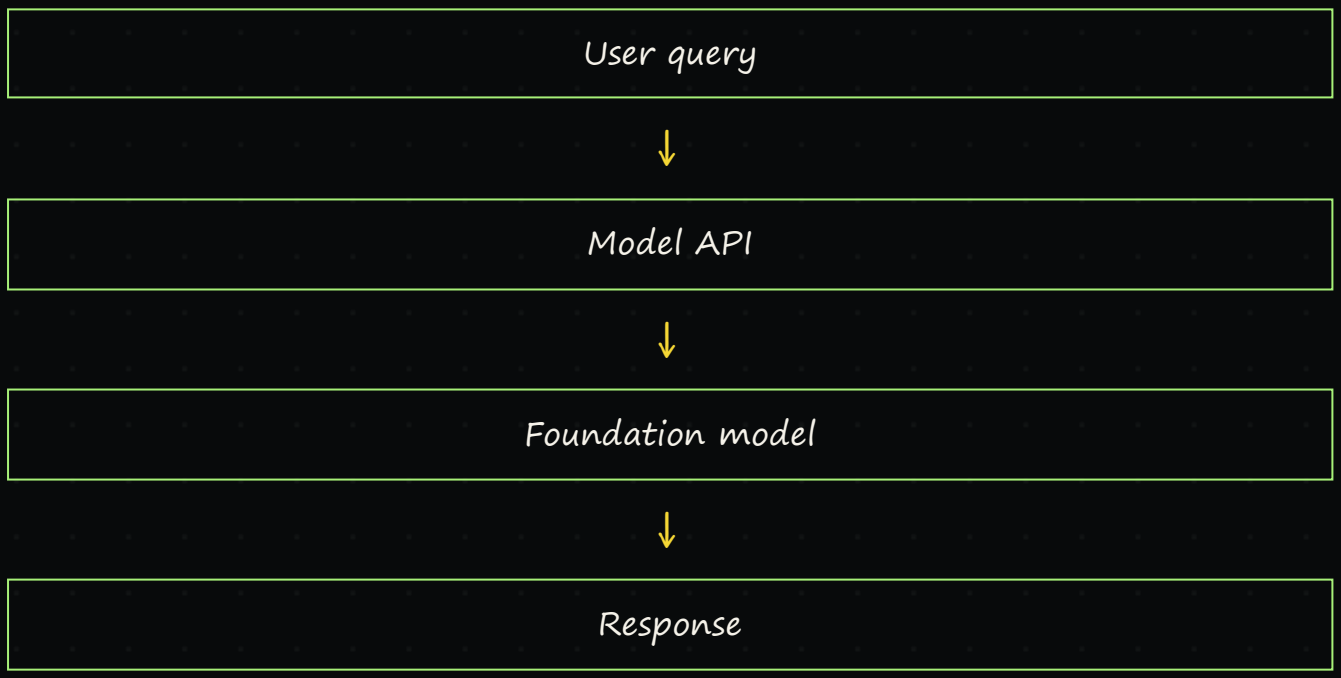
Architecture & User Feedback

Build a reliable path from user request to useful response

A production AI application is more than a model API call. It needs grounded context, safety controls, model routing, gateways, caches, tool workflows, observability, and a feedback loop that improves the product without amplifying bias.

01 Start with the Simple Architecture

Baseline request flow



This is a useful starting point for a prototype, but it does not yet manage context, safety, model choice, latency, actions, or production monitoring.

Why production needs more

- A bare prompt may lack facts and encourage unsupported answers.
- There are no protections for sensitive input or harmful prompts.
- Complex products may need different models and tools for different requests.
- Repeated retrieval and generation can add avoidable latency and cost.
- Some tasks need iterative agent behavior or controlled write actions.
- Without monitoring and logs, failures and drift are hard to diagnose.

---

02 Construct and Ground Context

Context construction

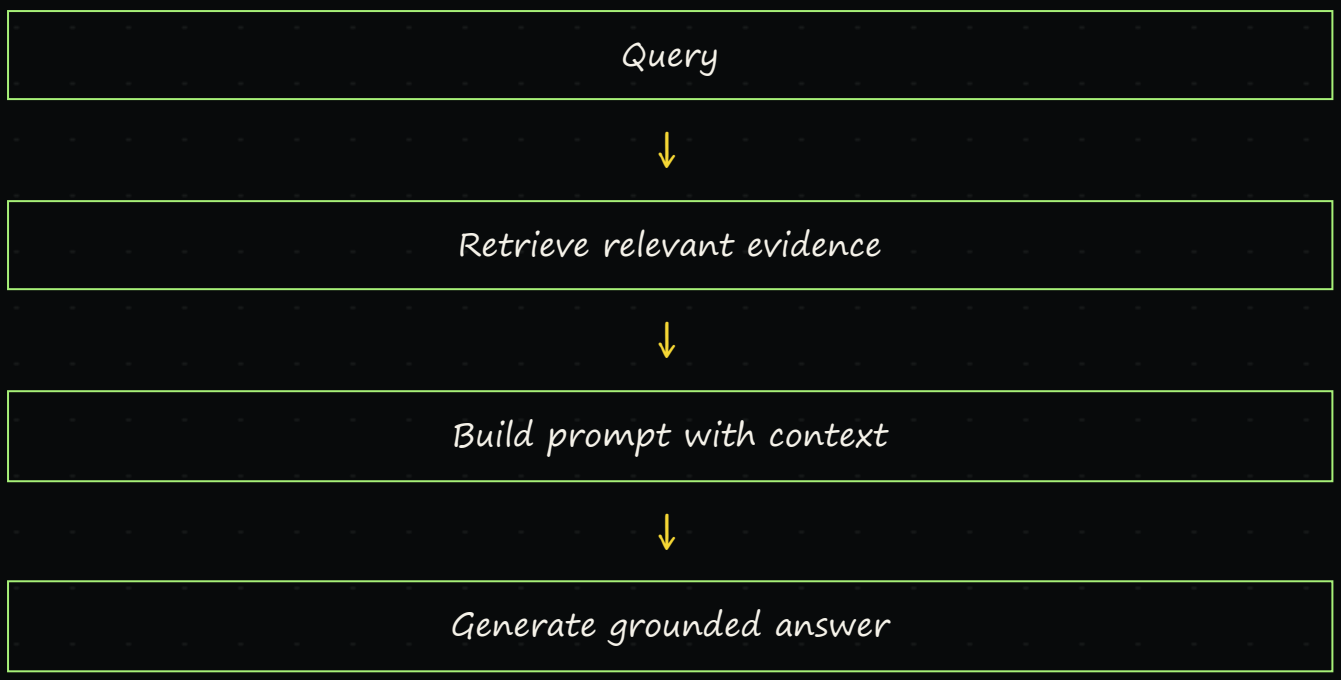
Context construction supplies the model with relevant information beyond the raw user query. It acts like feature engineering for foundation models: select and shape useful evidence before generation.

Common patterns include RAG, query rewriting, conversation history, and tools or agents that retrieve information.

Possible data sources and read actions

- Documents, structured tables, and product records.
- Conversation history and user-approved memory.
- Vector search over semantic embeddings.
- SQL queries for structured data.
- Web search or other approved retrieval tools.

Grounded request flow



Grounding can reduce unsupported answers, but the application must still evaluate retrieval relevance, permissions, freshness, and whether the model used the evidence correctly.

---

03 Input and Output Guardrails

Input guardrails

Inspect or transform incoming requests before they reach a model, especially an external API.

- Detect and block or redact sensitive personal or company information when policy requires it.
- Protect against jailbreak attempts and prompt injection.
- Apply access checks and validate that the request is in scope.

Masking and unmasking sensitive values

Where policy permits computation on masked data, replace sensitive values locally with opaque tokens and keep a protected token-to-value map. Send only the masked request; restore values only in an authorized local step.

> Private value → local token map → masked request to model → controlled local restoration

Masking does not replace consent, access control, retention rules, or a review of whether sending transformed data is permitted.

Output guardrails

- **Quality:** malformed JSON, invalid schema, unsupported or inconsistent claims.
- **Safety/security:** toxic content, privacy leakage, or disallowed instructions.
- **Refusal behavior:** check both missed refusals and unnecessary false refusals.
- **Actions:** review requests to send messages, update records, or perform consequential operations.

Reliability, latency, and streaming

Guardrails add components to the request path and can increase latency. Output checks may need to inspect multiple tokens or the completed response, which complicates immediate token streaming. Use risk-appropriate checks and clear retry or escalation policies.

Human approval may be required for ambiguous or high-impact tool actions.

04 Model Router and Gateway

Router: choose the right path

A router directs a request to an appropriate model, tool, or human path. It can use an intent classifier to understand the request and a next-action predictor to choose among possible steps.

- Specialized smaller models may reduce serving cost for narrow tasks.
- Some requests should be routed to a human or a higher-capability model.
- Routing may happen before retrieval to determine which data sources and permissions apply.

Context length is a routing constraint

Context construction can make a short query much longer. A specialized model may be more accurate but have a smaller context window; a larger model may accept the full context but cost more or be less specialized.

Choose among trimming, summarizing, retrieving less, or selecting a model with more context based on expected information loss, quality, latency, and cost.

Gateway: one managed interface

A model gateway provides a common interface to self-hosted models and external providers. It centralizes a model catalog, access control, credential handling, usage limits, and cost policies.

Gateway operational duties

- Route API requests and normalize provider-specific details.
- Track usage, budgets, and API limits.
- Provide fallback behavior when a model or provider fails.
- Support load balancing, logging, and analytics.

05 Caching for Latency and Cost

Exact cache

Reuse a stored result when the incoming request matches a previously seen request under the cache key. It is precise but misses paraphrases and semantically equivalent queries.

Cache retrieval results or responses close to the application for quick access; use appropriate in-memory or low-latency storage.

Semantic cache

Represent queries as embeddings and reuse an answer when a new query is sufficiently similar to a cached query. This can raise the hit rate for paraphrases but may return an inappropriate answer if the threshold, embedding, or context is wrong.

Test similarity thresholds against real queries. Semantic cache quality depends on embeddings, vector search, and a reliable similarity metric.

Cache policy choices

- **LRU:** evict least recently used items.
- **LFU:** evict least frequently used items.
- **FIFO:** evict oldest inserted items.
- Set expiration/TTL and decide which requests are safe and useful to cache.

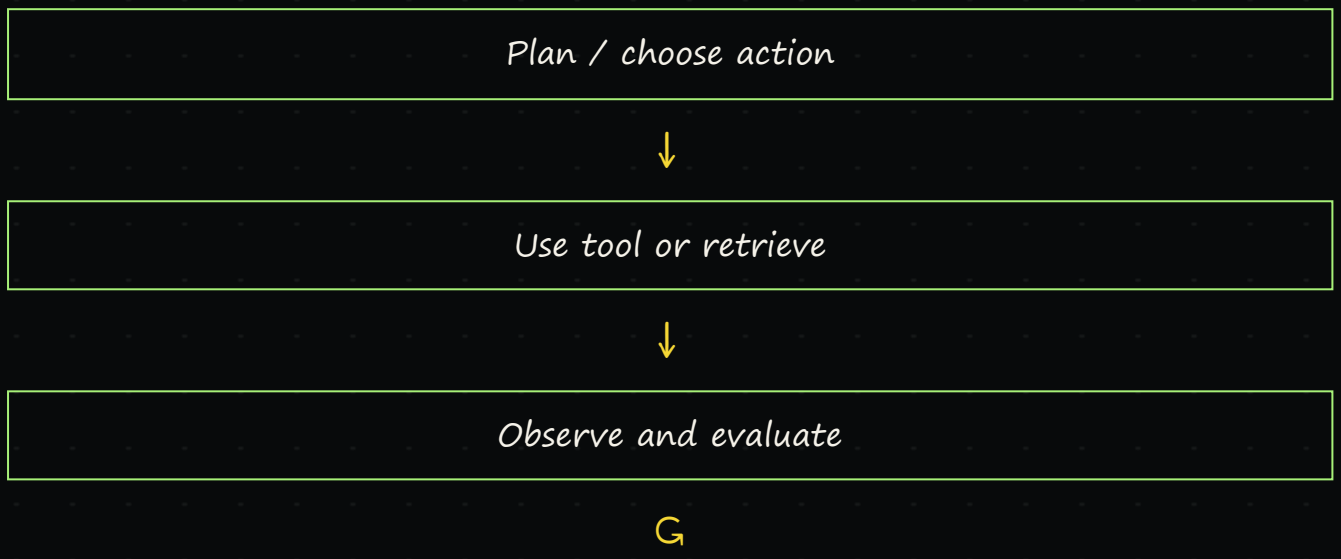
Cache both carefully

Applications may cache retrieved context as well as final responses. Respect user identity, authorization, privacy, and freshness boundaries so one user cannot receive another user's data and stale results do not persist past policy.

06 Agentic Patterns and Orchestration

Agent loop

A single-pass system returns its first response. An agentic workflow can evaluate an intermediate result, retrieve more context, choose another action, and refine the response until a stopping condition is met.



Set limits on steps, time, tool permissions, and cost to prevent uncontrolled loops.

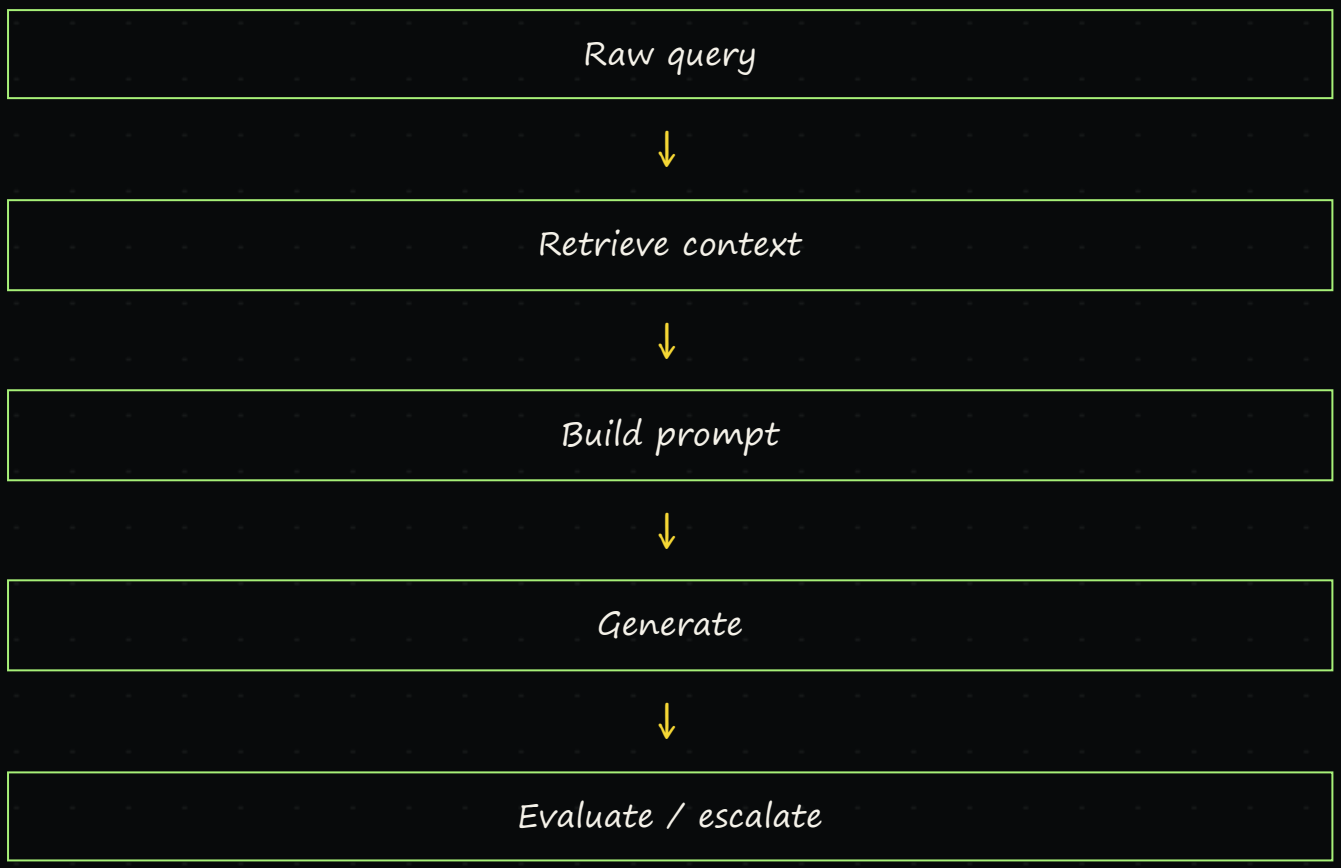
Read and write actions

Read actions fetch information. Write actions can change external state, such as updating an order or sending an email, and therefore require stricter authorization and confirmation.

Separate the model's proposed action from the trusted component that validates and executes it.

Orchestration chains

Frameworks such as LangChain can connect model calls, retrievers, tools, and evaluators. A simple chain might be:



Assess an orchestration approach for integration and extensibility, support for complex or parallel pipelines, usability, performance, and scalability.

---

07 Monitoring, Observability, and Metrics

Monitoring vs. observability

Monitoring tracks defined signals and alerts when they cross limits. **Observability** uses metrics, logs, traces, and system context to investigate why a behavior occurred.

Operational measures

- **MTTD:** mean time to detect a problem.
- **MTTR:** mean time to respond to or resolve a detected problem.
- **Change failure rate:** share of deployments that cause failures requiring corrective work.

Lower detection and resolution times indicate faster incident response; definitions should be consistent within the team.

Logs, metrics, and traces

- **Metrics:** aggregate signals over time for trends and alerts.
- **Logs:** event records useful for inspecting individual failures.
- **Traces:** a request's path through services, models, retrieval, and tools.

Design metrics around failures

- Grounding/faithfulness and hallucination indicators.
- Schema and format failure rates.
- Safety-trigger, refusal, and policy-bypass rates.
- Token usage, API cost, and latency such as TTFT and total response time.
- Output length when users need concise responses.
- RAG retrieval quality, context relevance, and precision.

Connect model metrics to product outcomes

Evaluation metrics should map to operational monitoring, and both should connect to business outcomes such as task completion, retention, or service quality. If production monitoring reveals a failure not covered by offline evaluation, update the evaluation set and criteria.

Watch for drift in user behavior, system prompts, business needs, underlying models, and data. A strong benchmark score is not a substitute for observing the live product.

08 User Feedback and the Data Flywheel

Explicit and implicit signals

- **Explicit:** ratings, written comments, or selecting a preferred answer.
- **Implicit:** edits, corrections, early exits, repeated attempts, conversation length, or the kinds of tasks a user returns to.

These signals can reveal friction and guide product or model iteration, but behavior does not always reveal intent. Long sessions, for example, may indicate engagement or difficulty.

Choose when to ask

Collect feedback at useful moments, such as after a task, when the system has low confidence, or after a failure or correction. A pairwise choice can be easier than asking users to assign an abstract score.

Ask only when the signal is useful and the interaction cost is reasonable.

Feedback has bias

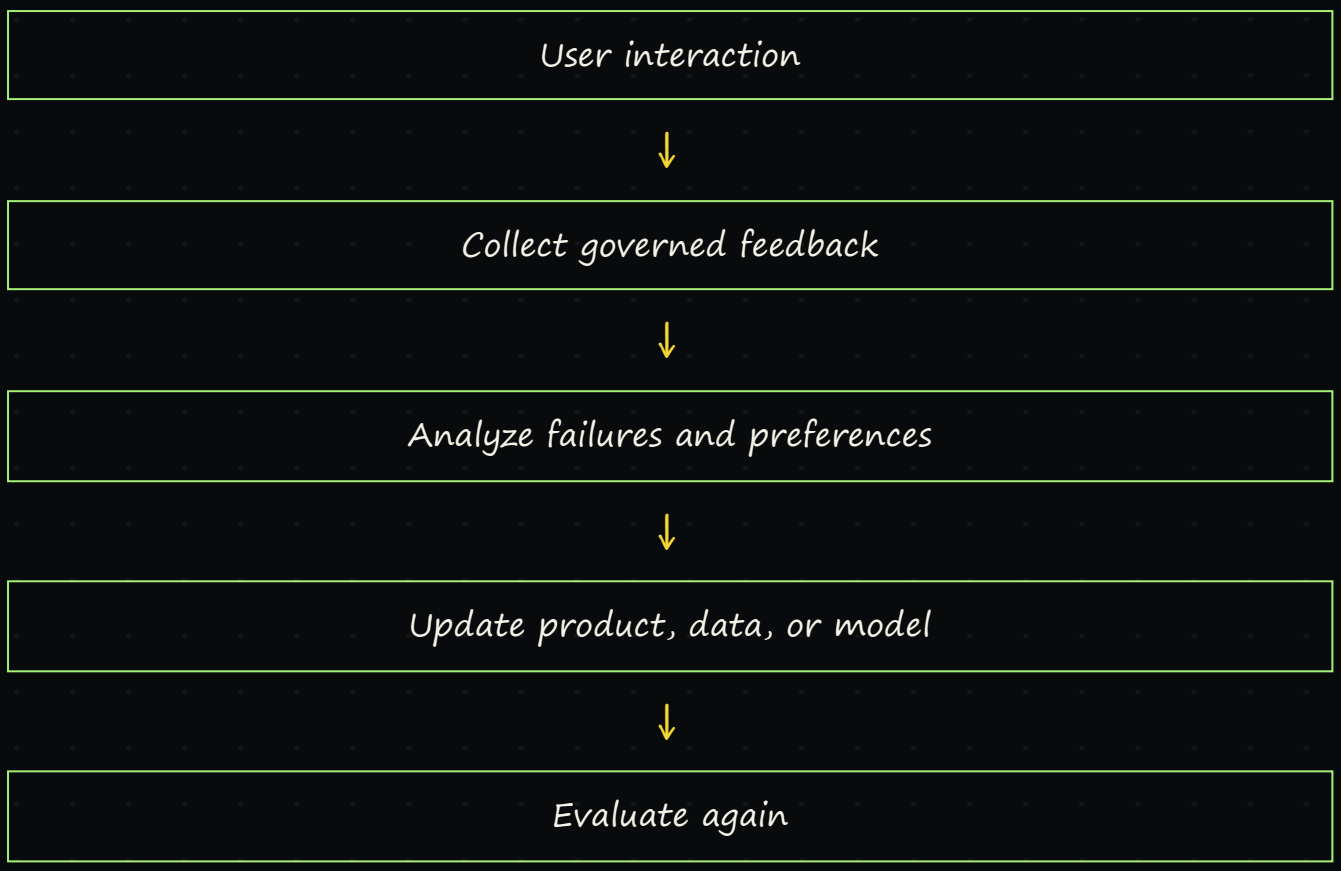
- Users may click randomly or without reading carefully.
- Position and recency can influence which answer is selected.
- Different users may have different preferences and expertise.
- Feedback may overrepresent frequent users or popular tasks.

Degenerate feedback loops

If a system shows more of what users already select, it may narrow its behavior and amplify existing preferences or bias. Training on approval signals can also encourage sycophancy: agreeing or flattering users instead of being accurate.

Preserve diversity, examine subgroup effects, and balance preference signals with correctness, safety, and expert review.

Close the loop responsibly



A data flywheel creates a product advantage only when data collection is privacy-aware, representative, and tied to measurable improvements. Feedback is an important signal, not an instruction to optimize blindly for clicks.

Prepared from the supplied Chapter 10 transcript. Feedback and monitoring metrics should be adapted to the product's policies and user context.